

Rapport du pipeline de données ParaMed

Table des matières

1	Introduction	3
2	Structure du projet	3
3	Collecte des données (Scraping)	3
3.1	Configuration des catégories	3
3.2	Scraping <i>parapharma.ma</i>	3
3.3	Scraping <i>universparadiscount.ma</i>	4
4	Nettoyage et transformation des données	4
4.1	Normalisation des noms	4
4.2	Extraction des marques et des formats	4
4.3	Nettoyage des prix, disponibilité et mappage des catégories	5
4.4	Fusion et nettoyage	5
5	Appariement des produits	5
6	Orchestration du pipeline	6
7	Tableau de bord et analyses	6
8	Conclusion et perspectives	7

1 Introduction

Le projet *ParaMed* a pour objectif de créer un ensemble de données exhaustif de produits parapharmaceutiques vendus sur deux sites de e-commerce marocains—parapharma.ma et universparadiscount.ma—et d’en tirer des informations exploitables. Le dépôt `paraMed_analysis` contient un paquet Python remanié (`paraMed_pipeline`) qui unifie la logique de scraping, de nettoyage et d’appariement en modules réutilisables. Selon le fichier README, le dépôt original se composait de plusieurs scripts autonomes ; la nouvelle structure centralise la configuration, les utilitaires de nettoyage et le point d’entrée de l’orchestration (see `paraMed_pipeline/README.md`, lines 255–289).

Dans ce rapport, nous décrivons en détail chaque étape du pipeline—from la collecte des données via le scraping web jusqu’à la construction des tableaux de bord finaux—and mettons en évidence les choix de conception et les principales conclusions. Les citations renvoient à des lignes spécifiques du code source du projet.

2 Structure du projet

Le pipeline est organisé en un petit nombre de modules. Le module `config.py` centralise les paramètres globaux tels que les noms des modèles d’embedding, les seuils de similarité et les listes de catégories et de marques connues (see `paraMed_pipeline/config.py`, lines 0–8) (see `paraMed_pipeline/config.py`, lines 16–29). Les modules de scraping dans `pipeline/scrapers` récupèrent les listes de produits sur chaque site. Le paquet `utils` contient des utilitaires de nettoyage (normalisation des chaînes, extraction des marques et des formats, mappage des catégories, etc.) ainsi qu’un assistant pour la base de données (see `paraMed_pipeline/pipeline/utils/db.py`, lines 0–8). Le module `transform.py` fusionne les documents bruts et applique la logique de nettoyage (see `paraMed_pipeline/pipeline/transform.py`, lines 0–16). Un moteur d’appariement basé sur des embeddings dans `matcher.py` identifie les produits dupliqués entre les sites (see `paraMed_pipeline/pipeline/matcher.py`, lines 0–9). Enfin, le script `main.py` orchestre le scraping, la fusion, le stockage et l’appariement en un seul point d’entrée (see `paraMed_pipeline/pipeline/main.py`, lines 0–13).

Tout au long de ce rapport, lorsque nous nous référons à des fonctions ou à des modules, nous incluons des notes de bas de page pointant vers les lignes pertinentes du dépôt.

3 Collecte des données (Scraping)

3.1 Configuration des catégories

Avant de lancer le scraping, le pipeline définit des listes de catégories pour chaque site. Les variables `config.PARAPHARMA_CATEGORIES` et `config.UNIVERS_CATEGORIES` fournissent des paires nom/URL pour les pages de catégories (see `paraMed_pipeline/config.py`, lines 40–70). Par exemple, la liste Parapharma comprend des catégories telles que « Visage », « Maquillage », « Corps » et « Solaire », tandis que la liste Univers contient « Homme », « Cheveux » et « Visage ». Ces listes peuvent être étendues ou remplacées à l’exécution, permettant d’ajouter de nouvelles catégories sans modifier la logique de scraping.

3.2 Scraping *parapharma.ma*

Le scraper `parapharma.py` parcourt des pages de catégories paginées. Son docstring explique qu’il récupère des listings de produits pour chaque catégorie et utilise un schéma cohérent (see `paraMed_pipeline/pipeline/scrapers/parapharma.py`, lines 0–8). La fonction `scrape_category_page` construit des URL avec un paramètre de pagination, requête la page, analyse les cartes produit

avec BeautifulSoup et extrait des champs tels que le nom du produit, le prix, la remise, l'état de disponibilité, l'URL du produit et l'URL de l'image (see `paraMed_pipeline/pipeline/scrapers/parapharma.py`, lines 21–43). Si une remise est présente, le scraper calcule le prix initial (prix + remise) et définit un indicateur indiquant si le produit est en promotion (see `paraMed_pipeline/pipeline/scrapers/parapharma.py`, lines 71–85). La disponibilité est dérivée des indicateurs CSS (« rupture » contre « disponible »), et la pagination s'arrête lorsqu'aucun produit n'est trouvé ou lorsqu'une limite maximale de pages est atteinte. Une fonction d'assistance `scrape_all` parcourt la liste configurée des catégories et concatène les résultats (see `paraMed_pipeline/pipeline/scrapers/parapharma.py`, lines 119–145).

3.3 Scraping *universparadiscount.ma*

Le scraper `univers.py` reproduit la logique de Parapharma mais s'adapte à la structure des pages de Univers ParaDiscount. Son docstring indique qu'il parcourt les pages de catégories jusqu'à ce qu'aucun produit ne soit trouvé et produit des dictionnaires selon un schéma standard (see `paraMed_pipeline/pipeline/scrapers/univers.py`, lines 0–7). La fonction `scrape_category_page` construit des URL avec un paramètre `resultsPerPage`, analyse les éléments de produit, extrait les noms, les URL de produit, les URL d'image, les catégories et les prix (à la fois normaux et remisés) (see `paraMed_pipeline/pipeline/scrapers/univers.py`, lines 21–42). Elle détermine ensuite le prix final et la remise en fonction de la présence de prix réguliers et remisés (see `paraMed_pipeline/pipeline/scrapers/univers.py`, lines 86–110) et déduit la disponibilité à partir des indicateurs CSS (« rupture »/« disponible »). Comme le scraper Parapharma, une fonction `scrape_all` traite toutes les catégories configurées (see `paraMed_pipeline/pipeline/scrapers/univers.py`, lines 134–158).

4 Nettoyage et transformation des données

Les données brutes provenant des deux sites nécessitent une normalisation approfondie pour permettre l'appariement et l'analyse cross-site. Les fonctions de nettoyage sont implémentées dans `utils/cleaning.py`. Le docstring du module résume son objectif : il normalise les noms de produits, extrait les marques et les formats, nettoie les prix et les labels de disponibilité et mappe les catégories (see `paraMed_pipeline/pipeline/utils/cleaning.py`, lines 0–16).

4.1 Normalisation des noms

La fonction `clean_name` met en minuscules les chaînes, supprime les accents et les caractères non alphanumériques, remplace les tirets et les traits de soulignement par des espaces, concatène les nombres avec les unités et réduit les espaces (see `paraMed_pipeline/pipeline/utils/cleaning.py`, lines 53–83). Ceci permet d'obtenir une représentation standardisée améliorant la détection de doublons. Un assistant connexe `clean_name_from_image_url` récupère les noms tronqués dans les annonces Parapharma en extrayant un slug de l'URL de l'image du produit et en transformant les tirets en espaces (see `paraMed_pipeline/pipeline/utils/cleaning.py`, lines 86–117). Le pipeline utilise cet assistant lorsque le nom brut se termine par des points de suspension pour s'assurer que les produits aux noms abrégés puissent toujours être appariés (see `paraMed_pipeline/pipeline/transform.py`, lines 70–84).

4.2 Extraction des marques et des formats

La fonction `extract_size` recherche dans le nom nettoyé des éléments comme « 200ml » ou « 50mg » et retourne la première correspondance (see `paraMed_pipeline/pipeline/utils/cleaning.py`, lines 119–134). Pour extraire les marques, `extract_brand` compare le début du nom nettoyé avec une liste de marques connues (définie dans `config.KNOWN_BRANDS`) et applique des heuristiques lorsqu'aucune marque connue ne correspond (see `paraMed_pipeline/pipeline/utils/cleaning.py`,

lines 135–162). Les mots de la liste noire des marques (par exemple, « applicateur », « brosse ») sont ignorés pour éviter de considérer des termes courants comme des marques (see `paraMed_pipeline/config.py`, lines 94–109).

4.3 Nettoyage des prix, disponibilité et mappage des catégories

L’assistant `clean_price` supprime les caractères non numériques d’une chaîne de prix, remplace les virgules par des points et convertit le résultat en nombre à virgule flottante (see `paraMed_pipeline/pipeline/utils/cleaning.py`, lines 222–247). Les codes de disponibilité provenant de différentes sources (« `in_stock` », « `disponible` », « `out_of_stock` », « `rupture` ») sont normalisés en « `disponible` » ou « `indisponible` » (see `paraMed_pipeline/pipeline/utils/cleaning.py`, lines 195–220). Les libellés de catégories extraits des sites présentent souvent de nombreuses variantes ; la fonction `map_category` normalise la chaîne d’entrée à l’aide de `clean_name` et recherche des clés prédéfinies dans le dictionnaire `category_mapping`. Si aucune correspondance n’est trouvée, la catégorie « `Autres` » est retournée (see `paraMed_pipeline/pipeline/utils/cleaning.py`, lines 249–276). Le dictionnaire de mapping est défini dans `utils/category_mapping.py` et mappe des chaînes telles que « `gommages visage` » ou « `apres soleil` » à des catégories canoniques (see `paraMed_pipeline/pipeline/utils/category_mapping.py`, lines 0–10).

4.4 Fusion et nettoyage

La phase de transformation est prise en charge par la fonction `merge_and_clean` dans `transform.py`. Son docstring indique qu’elle fusionne des dictionnaires de produits bruts provenant de plusieurs sources et les normalise dans un schéma unifié, appliquant le nettoyage, l’extraction des marques et des formats, la logique de prix/remise, la normalisation de la disponibilité et le mappage des catégories (see `paraMed_pipeline/pipeline/transform.py`, lines 0–16). La fonction accepte des listes de produits extraits de Parapharma et d’Univers et peut dédupliquer les produits par site et par nom nettoyé. Elle calcule le prix initial et la remise, détermine si un produit est en promotion et retourne une liste de dictionnaires comportant les champs `site`, `product_url`, `category`, `main_category`, `name`, `clean_name`, `brand`, `size`, `price`, `original_price`, `discount`, `is_discounted`, `availability`, `image_url` et `scraped_at` (see `paraMed_pipeline/pipeline/transform.py`, lines 40–66). La détection des doublons s’effectue en suivant les paires (site, nom nettoyé) déjà vues et en sautant les doublons (see `paraMed_pipeline/pipeline/transform.py`, lines 70–88). Les noms récupérés à partir des URL d’image sont appliqués spécifiquement aux entrées Parapharma dont le nom brut se termine par « `...` » (see `paraMed_pipeline/pipeline/transform.py`, lines 70–84).

5 Appariement des produits

Une fois les données nettoyées, les produits des deux sites sont comparés afin d’identifier les doublons ou les correspondances proches. Le moteur d’appariement utilise des embeddings de phrases pour calculer la similarité sémantique entre les descriptions de produits. Le module `matcher.py` décrit un moteur d’appariement simple basé sur des embeddings (see `paraMed_pipeline/pipeline/matcher.py`, lines 0–9). Les produits sont regroupés par marque et par format ; pour chaque produit Parapharma, un embedding de la marque, du nom nettoyé et du format est calculé et comparé aux candidats Univers ayant la même marque et le même format (see `paraMed_pipeline/pipeline/matcher.py`, lines 35–62). Les scores de similarité cosinus sont calculés à l’aide d’un modèle spécifié par `config.EMBEDDING_MODEL` (par défaut « `paraphrase-multilingual-MiniLM-L12-v2` ») (see `paraMed_pipeline/config.py`, lines 16–29). Les correspondances dont le score de similarité dépasse le seuil défini dans `config.SIMILARITY_THRESHOLD` (0,90) sont conservées (see `paraMed_pipeline/config.py`, lines 23–29). Si un produit Parapharma possède

plusieurs candidats au-dessus du seuil, l'algorithme conserve uniquement la correspondance avec la plus haute similarité (see `paraMed_pipeline/pipeline/matcher.py`, lines 95–107).

6 Orchestration du pipeline

Le script `main.py` fournit une fonction de haut niveau `run_pipeline` qui orchestre le scraping, le nettoyage, le stockage et l'appariement. Son docstring indique qu'exécuter `python -m paraMed_pipeline.pipeline.main` lance le pipeline complet et que des variables d'environnement MongoDB doivent être configurées (see `paraMed_pipeline/pipeline/main.py`, lines 0–13). La fonction commence par extraire les produits de Parapharma et d'Univers en utilisant `scrape_all` pour chaque site. Le nombre maximum de pages par site peut être spécifié ; par défaut, le pipeline récupère jusqu'à 156 pages pour Parapharma et une page pour Univers (see `paraMed_pipeline/pipeline/main.py`, lines 27–40). Il fusionne ensuite les données brutes nettoyées, enregistre les documents nettoyés dans une collection MongoDB nommée `para_univer_merged`, effectue l'appariement et stocke les correspondances dans une collection `matches` (see `paraMed_pipeline/pipeline/main.py`, lines 41–75). Les assistants de base de données dans `utils/db.py` lisent les paramètres de connexion à partir des variables d'environnement et renvoient des objets client et collection MongoDB (see `paraMed_pipeline/pipeline/utils/db.py`, lines 19–34) (see `paraMed_pipeline/pipeline/utils/db.py`, lines 37–78).

7 Tableau de bord et analyses

Après que les données nettoyées et les correspondances ont été stockées dans MongoDB, un tableau de bord Power BI (`dashboard.pdf`) visualise les métriques clés et permet une exploration interactive. Le texte extrait du rapport indique les statistiques de haut niveau suivantes :

- **Nombre total de produits.** L'ensemble de données contient environ 22,7 k références uniques sur les deux sites. Le site Parapharma domine l'inventaire, reflétant une sélection plus large.
- **Marques et catégories uniques.** On recense environ 1 374 marques uniques et 203 catégories uniques. Cette diversité suggère une forte diversité de produits.
- **Disponibilité des produits.** Environ 87 % des produits sont signalés comme disponibles (« disponible ») tandis qu'environ 13 % sont en rupture de stock. La répartition de la disponibilité est visualisée sous forme de graphique à barres. Une analyse par catégorie révèle que certaines catégories, comme « Bébé & Maternité », présentent des proportions de ruptures plus élevées.
- **Promotions.** Environ un quart des produits bénéficient de remises (le tableau de bord indique qu'environ 24,7 % des produits sont en promotion). Comparer les articles remisés et non remisés donne un aperçu des stratégies de prix.
- **Distribution des prix.** Un histogramme montre que la plupart des produits sont bon marché (inférieurs à 0,4 k MAD) mais une longue traîne d'équipements para-médicaux de grande valeur augmente la moyenne. Le tableau de bord permet de filtrer par catégorie et par marque pour visualiser les distributions de prix plus finement.
- **Analyse par catégorie.** Une autre vue affiche le prix moyen et le nombre de produits par catégorie de premier niveau. Par exemple, les catégories telles que « Para-médical » et « Compléments / Santé » présentent des prix moyens plus élevés que des catégories de tous les jours comme « Hygiène » ou « Corps ». Le mappage des catégories décrit dans la Section 4 sous-tend cette analyse.
- **Produits phares.** Un tableau répertorie les produits individuellement avec leur prix, leur remise et leur marque. Les utilisateurs peuvent filtrer par catégorie, marque ou site pour identifier les meilleures ventes ou les remises élevées. La présence de produits Parapharma et Univers permet des comparaisons côte à côte.

Ces éléments montrent comment l'ensemble de données nettoyé peut être exploré de manière interactive. La conception modulaire du pipeline rend facile la mise à jour des données et la régénération du tableau de bord lorsque de nouveaux produits apparaissent ou que d'autres sites sont ajoutés.

8 Conclusion et perspectives

Le pipeline *ParaMed* intègre avec succès le scraping web, la normalisation textuelle, l'appariement basé sur des embeddings et l'analyse visuelle en un flux reproductible. En encapsulant la configuration dans un module dédié et en exposant des fonctions claires pour chaque étape, le projet gagne en clarté et en extensibilité (see `paraMed_pipeline/README.md`, lines 255–289). Les tableaux de bord finaux résument plus de 22 000 produits sur deux sites e-commerce, couvrant plus d'un millier de marques et des centaines de catégories. Le nettoyage des données garantit que les noms, prix, étiquettes de disponibilité et catégories sont cohérents, permettant un appariement et une analyse précis.

Il existe plusieurs pistes pour la suite :

- **Découverte dynamique des catégories.** Actuellement, le pipeline utilise des listes statiques de catégories définies dans le fichier de configuration. Extraire dynamiquement les pages de catégories pourrait traiter automatiquement les nouvelles catégories ajoutées aux sites.
- **Amélioration de l'appariement.** Le moteur d'appariement se base sur les noms de produits, les marques et les formats. L'intégration des descriptions ou des images pourrait améliorer la précision de l'appariement, et ajuster le seuil de similarité par catégorie pourrait équilibrer précision et rappel.
- **Sources supplémentaires.** La conception modulaire du scraper permet d'ajouter de nouveaux sites de e-commerce en implémentant un nouveau module de scraping et en mettant à jour le code d'orchestration (see `paraMed_pipeline/README.md`, lines 320–326). Étendre le pipeline augmenterait la couverture et permettrait une analyse du marché à l'échelle.
- **Analyses renforcées.** L'intégration d'analyses statistiques avancées ou de modèles d'apprentissage automatique pourrait identifier des anomalies de prix, prévoir les ruptures de stock ou recommander des regroupements de produits.

Le code et ce rapport constituent ensemble une base solide pour maintenir et étendre le pipeline de données ParaMed.